

PulseOps

Buku Panduan Komprehensif Arsitektur, Pengujian Otomatis, Orkestrasi Kubernetes Zero-Trust, SLO Google SRE, dan Handover Rekayasa

PENULIS & PENGEMBANG

Yusuf Jaelani

TEKNOLOGI UTAMA

Go 1.23, Pure-Go SQLite, K8s, Prometheus, Grafana

TARGET PEMANTAUAN UTAMA

portfolioyusufjaelani.vercel.app

VERSI RILIS & LISENSI

v1.0.0 (Production Grade) • MIT License

STRUKTUR DOKUMEN PANDUAN

1. Ikhtisar & Nilai Bisnis	Hal 2	2. Arsitektur & Alur Permintaan	Hal 2
3. Struktur Kode & Peta Berkas	Hal 3	4. Panduan Memulai Cepat	Hal 3
5. Pengujian Otomatis & DevSecOps	Hal 4	6. Task Runner (Makefile)	Hal 4
7. Referensi Environment Variables	Hal 5	8. Referensi API & Telemetry	Hal 5
9. Panduan Handover & Troubleshooting	Hal 6	10. Panduan Kontribusi & Standar PR	Hal 7
11. Keamanan & Pengerasan Produksi	Hal 7	12. Deployment K8s & Terraform	Hal 8

1. Ikhtisar & Nilai Bisnis

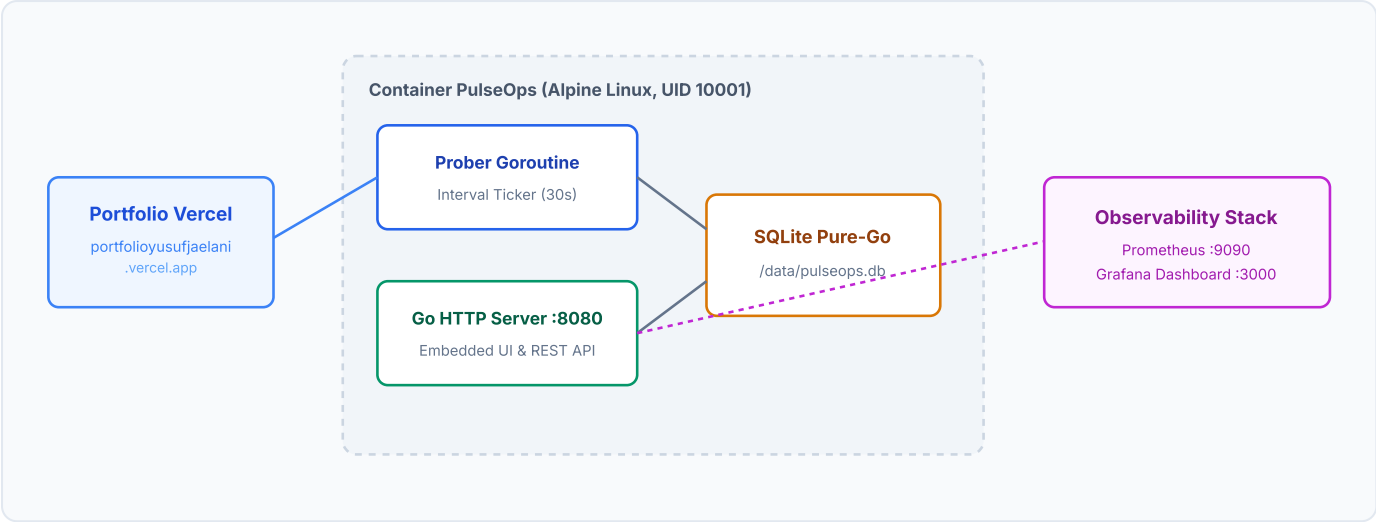
PulseOps dirancang khusus untuk memecahkan kesenjangan observabilitas pada arsitektur web modern (seperti Vercel, JAMStack, dan microservices). Banyak tim menghadapi situasi di mana edge CDN menyajikan cache kadaluwarsa, masa aktif sertifikat SSL habis tanpa peringatan, atau latensi jaringan meningkat tajam tanpa terdeteksi oleh dashboard konvensional.

Karakteristik Kunci Sistem

- **Single Binary Statis:** Web UI (Tailwind CSS + Chart.js) tersemat langsung di dalam binary Go via `//go:embed`. Tidak membutuhkan runtime Node.js atau reverse proxy tambahan.
- **Portabilitas Tanpa CGO:** Menggunakan pure-Go SQLite (`modernc.org/sqlite`) yang menghilangkan kebutuhan compiler GCC / C-libraries, menjamin portabilitas multi-arsitektur (AMD64 & ARM64).
- **Observabilitas Standar Enterprise:** Menyediakan metrik native OpenMetrics (`/metrics`), probe Kubernetes (`/healthz`, `/readyz`), serta aturan SLO / Error Budget Google SRE.
- **Footprint Minimalis:** Docker multi-stage menghasilkan image produksi berukuran hanya **~8.5MB** berjalan di bawah user non-root (`UID 10001`).

2. Arsitektur & Alur Permintaan

Secara operasional, PulseOps menjalankan loop prober goroutine otonom yang mengeksekusi handshake HTTP/TLS, mencatat header caching Vercel (`x-vercel-cache`), menghitung SLA ketersediaan, serta mengekspos metrik ke stack Prometheus dan Grafana.



3. Struktur Kode & Peta Berkas

Kode disusun mengikuti prinsip standar Go Project Layout yang bersih dan mudah dinavigasi:

```
.
├── .github/workflows/ci.yml      # CI/CD: gofmt, test coverage, Trivy scanner, GHCR multi-arch
├── cmd/server/main.go           # Entrypoint aplikasi & penanganan graceful shutdown
├── deploy/
│   ├── grafana/provisioning/    # Provisioning otomatis datasource & dashboard SRE Grafana
│   ├── k8s/                    # Manifests K8s (Deployment, Service, PVC, NetworkPolicy, Ingress)
│   ├── prometheus/             # Aturan alert SRE & formula Google SRE SLO/Error Budget
│   └── terraform/              # Declarative Infrastructure as Code (Docker container & volume)
├── internal/
│   ├── config/config.go         # Pengurai variabel lingkungan (PORT, DB_PATH, interval)
│   ├── handler/                # Controller HTTP, /metrics exporter, /healthz, /readyz
│   ├── model/target.go         # Domain model: Target, ProbeLog, TargetSummary
│   ├── prober/prober.go        # Engine probing jaringan (HTTP latency, SSL handshake, CDN cache)
│   └── store/store.go           # Database access layer SQLite pure-Go (DDL, cascade delete, SLA)
├── web/                        # Antarmuka Single-Page Application (HTML, JS, Tailwind, Chart.js)
├── Dockerfile                  # Multi-stage build aman (golang:1.23-alpine → alpine:3.20)
├── docker-compose.yml          # Orkestrasi lokal: PulseOps + Prometheus + Grafana
├── Makefile                    # Automation runner untuk test, build, dan deployment
└── README.md                   # Sumber kebenaran dokumentasi tunggal proyek
```

4. Panduan Memulai Cepat & Navigasi Antarmuka

A. Menjalankan via Docker Compose (Direkomendasikan)

Cukup jalankan satu perintah untuk mengaktifkan seluruh ekosistem observabilitas:

```
docker compose up -d --build
```

LAYANAN	URL AKSES	DESKRIPSI & CARA PENGGUNAAN
Dashboard PulseOps	<code>http://localhost:8080</code>	Antarmuka manusia visual (Tailwind CSS + Chart.js). Menampilkan status real-time, latensi, sisa hari sertifikat SSL, dan grafik interaktif.
Prometheus UI	<code>http://localhost:9090</code>	Tampilan awal berstatus <i>"No data queried yet"</i> . Masukkan kueri PromQL (contoh: <code>pulseops_target_latency_ms</code>), lalu klik Execute . Periksa kesehatan target pada menu Status → Targets .
Grafana SRE	<code>http://localhost:3000</code>	Terbuka langsung tanpa login. Dashboard <i>"PulseOps - Telemetry & Uptime Dashboard"</i> sudah terpasang otomatis (*pre-provisioned*) menyajikan 5 panel metrik live.
Machine Probes	<code>/healthz</code> & <code>/readyz</code>	Endpoint JSON (<code>{"status": "ok"}</code>) untuk pemeriksaan liveness/readiness otomatis oleh Kubernetes atau Load Balancer.
Metrik OpenMetrics	<code>/metrics</code>	Data telemetri berformat teks mentah untuk di-scrape oleh server Prometheus.

5. Pengujian Otomatis & Pipeline DevSecOps CI/CD

Setiap commit yang didorong ke branch `main` memicu pipeline GitHub Actions (`.github/workflows/ci.yml`) yang terdiri dari tahapan verifikasi ketat:

- **Code Quality Check:** Memverifikasi bahwa sintaksis kode mematuhi standar `gofmt` .
- **Automated Testing Suite:** Menjalankan seluruh unit dan integration test mencakup `internal/store` (relasi SQLite, cascade delete), `internal/prober` (mock HTTP/TLS server, timeout boundary), dan `internal/handler` (REST API, exporter prometheus). Menghasilkan laporan cakupan kode (coverage).
- **Docker Smoke Test:** Membangun container dan menguji kesehatan endpoint `/healthz` di dalam runtime terisolasi.
- **DevSecOps Vulnerability Scan (Aqua Trivy):** Memindai image container dari kerentanan keamanan level `HIGH` dan `CRITICAL` sebelum diizinkan rilis.
- **Multi-Architecture Continuous Delivery:** Mengompilasi dan mempublikasikan image ke GitHub Container Registry (`ghcr.io/yusufj12/pulseops`) untuk arsitektur `linux/amd64` dan `linux/arm64` .

Cara Menjalankan Test Suite Secara Mandiri

Pengembang dapat memverifikasi pengujian tanpa menginstall Go di komputer lokal:

```
# Menggunakan task runner Makefile
make test

# Atau menggunakan Docker langsung
docker run --rm -v "${PWD}:/app" -w /app golang:1.23-alpine go test -v ./...
```

6. Task Runner Pengembang (Makefile)

PERINTAH	KEGUNAAN
<code>make test</code>	Menjalankan seluruh pengujian unit & integrasi di container terisolasi.
<code>make docker-build</code>	Membangun image container produksi <code>pulseops:latest</code> .
<code>make docker-compose-up</code>	Membangun dan menjalankan stack lengkap (PulseOps + Prometheus + Grafana) di background.
<code>make docker-compose-down</code>	Menghentikan seluruh container stack.
<code>make clean</code>	Membersihkan container dan menghapus persistent volume database.

7. Referensi Environment Variables

VARIABLE	DEFAULT	WAJIB?	DESKRIPSI OPERASIONAL
PORT	8080	Tidak	Port HTTP server mendengarkan koneksi.
DB_PATH	/data/pulseops.db	Tidak	Path lokasi penyimpanan berkas persistent SQLite.
PROBE_INTERVAL_SECONDS	30	Tidak	Interval periodik eksekusi daemon prober latar belakang.
DEFAULT_TARGET_URL	https://portfolioyusufjaelani.vercel.app	Tidak	Target default yang otomatis di-seed saat basis data pertama kali dibuat.
DEFAULT_TARGET_NAME	Yusuf Portfolio	Tidak	Label nama deskriptif untuk target default.

8. Referensi REST API, Alerting & Google SRE SLO

A. REST API Endpoints

- GET /api/targets : Mengambil daftar seluruh endpoint yang dipantau beserta ringkasan SLA dan probe terkini.
- POST /api/targets : Mendaftarkan target baru (menerima payload JSON `{"name": "...", "url": "..."}`).
- DELETE /api/targets/{id} : Menghapus target beserta seluruh riwayat log-nya secara bersih (cascade).
- POST /api/targets/{id}/probe : Memicu pemeriksaan jaringan secara instan tanpa menunggu interval daemon.
- GET /api/targets/{id}/history?limit=30 : Mengambil data historis latensi untuk visualisasi grafik.

B. Aturan Alerting SRE (`deploy/prometheus/alert.rules.yml`)

- TargetDown (Critical):** Memicu insiden saat `pulseops_target_up == 0` selama lebih dari 1 menit.
- HighLatencyWarning (Warning):** Memicu peringatan saat `pulseops_target_latency_ms > 1000ms` selama 2 menit.
- SSLExpirationWarning & Critical:** Peringatan dini saat masa aktif sertifikat SSL `< 14 hari` atau `< 3 hari`.

C. Standar Google SRE SLO & Error Budget (`deploy/prometheus/slo.rules.yml`)

- SLI Ketersediaan:** `pulseops:target_uptime_ratio:rate1h` mengukur rasio uptime aktual bergulir.
- Error Budget Burn Rate:** Menghitung laju konsumsi anggaran kegagalan berdasarkan target SLO **99.9%**.

- **ErrorBudgetFastBurn (Critical):** Mengeskalasi insiden darurat saat error budget terbakar lebih dari 14.4x laju normal.

9. Panduan untuk Engineer Penerus (Handover & Troubleshooting)

A. Rasional Arsitektur Pure-Go SQLite

Keputusan memilih driver `modernc.org/sqlite` alih-alih driver berbasis CGO (seperti `mattn/go-sqlite3`) didasarkan pada pertimbangan rekayasa keandalan:

- Memungkinkan binary dikompilasi dengan bendera `CGO_ENABLED=0` statis penuh tanpa ketergantungan pada `glibc` / `musl`.
- Dapat berjalan mulus di container Alpine minimalis tanpa perlu memasang paket `build-base` atau compiler GCC.
- Menghindari kerentanan keamanan memori tingkat C pada pustaka host.

B. Panduan Menambahkan Metrik Baru (Contoh: Time to First Byte / TTFB)

1. **Domain Model:** Tambahkan kolom `TTFBMs int64` pada struct `ProbeLog` di `internal/model/target.go`.
2. **Database:** Perbarui skema DDL tabel `probe_logs` pada metode `InitDB` di `internal/store/store.go`.
3. **Prober Engine:** Pasang `httptrace.ClientTrace` pada HTTP request di `internal/prober/prober.go` untuk mencatat waktu durasi `GotFirstResponseByte`.
4. **Exporter:** Daftarkan gauge baru `pulseops_target_ttfb_ms` di fungsi `handleMetrics` pada `internal/handler/handler.go`.
5. **Visualisasi:** Tambahkan komponen bar/chart pada `web/app.js` dan `web/index.html`.

C. Panduan Integrasi Alert Webhook (Discord / Slack / Telegram)

1. Tambahkan field konfigurasi `ALERT_WEBHOOK_URL` di `internal/config/config.go`.
2. Pada `internal/prober/prober.go`, bandingkan status probe saat ini dengan status log sebelumnya (deteksi transisi `UP → DOWN` atau `DOWN → UP`).
3. Jika terjadi transisi status, picu goroutine asinkron untuk mengirim HTTP POST payload JSON ke webhook URL.

D. Panduan Pemecahan Masalah Umum (Troubleshooting)

Daftar Solusi Isu Operasional

1. **Port Konflik (`address already in use: 8080/9090/3000`):** Ubah pemetaan port host pada `docker-compose.yml` (contoh: `"8081:8080"`).
2. **Permission Denied pada Volume SQLite (`/data/pulseops.db`):** Container berjalan dengan non-root `UID 10001`. Pastikan direktori lokal memiliki hak akses: `mkdir -p data && chown -R 10001:10001 data`.
3. **Prometheus Menampilkan "No data queried yet":** Ini adalah status normal saat kolom kueri kosong. Masukkan ekspresi PromQL (misal: `pulseops_target_latency_ms`) lalu klik tombol **Execute**.
4. **Grafana Sempat Menampilkan "No data":** Pastikan berkas `datasource.yml` memiliki konfigurasi `uid: prometheus`. Jalankan `docker compose restart grafana` dan refresh peramban (F5).

10. Panduan Kontribusi & Standar Pengembang

Untuk menjaga integritas dan kebersihan repositori, setiap kontributor wajib mematuhi standar berikut:

- **Prinsip YAGNI & Boring Code:** Jangan menambahkan dependensi pihak ketiga untuk tugas yang dapat diselesaikan oleh pustaka standar Go (`net/http` , `crypto/tls` , `log/slog`).
- **Format Pesan Commit Terstruktur (Conventional Commits):**
 - `feat: ...` • Penambahan fitur operasional baru
 - `fix: ...` • Perbaikan bug / isu kestabilan
 - `refactor: ...` • Penyempurnaan kode tanpa mengubah fungsi bisnis
 - `docs: ...` • Pembaruan dokumentasi atau panduan teknis
 - `test: ...` • Penambahan cakupan pengujian unit/integrasi
- **Checklist Wajib Sebelum Mengajukan Pull Request:**
 1. Seluruh test otomatis lulus 100%: `make test`
 2. Format kode rapi sesuai standar Go: `go fmt ./...`
 3. Image Docker berhasil dibangun tanpa error: `make docker-build`
 4. Tidak ada credential hardcoded atau tautan localhost statis pada tampilan antarmuka.
 5. Analisis SonarLint menghasilkan **0 Code Smells / 0 Warnings**.

11. Keamanan & Pengerasan Produksi (Security Hardening)

VEKTOR KEAMANAN	IMPLEMENTASI TEKNIS
Non-Root Execution	Container beroperasi di bawah user non-privileged <code>appuser:appgroup</code> dengan <code>UID:GID 10001:10001</code> . Hak istimewa root sepenuhnya ditiadakan.
Graceful Shutdown	Aplikasi menangkap sinyal sistem operasi <code>SIGINT</code> dan <code>SIGTERM</code> menggunakan batas waktu <code>context.WithTimeout(5s)</code> untuk memastikan koneksi HTTP aktif diselesaikan dan transaksi SQLite di-flush secara aman.
Client Timeout Boundaries	Prober jaringan dikonfigurasi dengan batas waktu ketat 10 detik dan batas redirect maksimal 5 hops untuk mencegah serangan Denial of Service (DoS) dan kebocoran goroutine.
Automount Token Disabled	Pada manifest Kubernetes, <code>automountServiceAccountToken</code> disetel ke <code>false</code> guna mencegah serangan pencurian token akses cluster API dari dalam pod.

12. Panduan Deployment Kubernetes, Terraform & Cloud

A. Kubernetes GitOps via Kustomize (Zero-Trust Security)

Manifests siap produksi telah tersedia di direktori `deploy/k8s/` dengan isolasi jaringan Zero-Trust:

```
# 1. Pratinjau resource yang digenerate oleh Kustomize
kubectl kustomize deploy/k8s

# 2. Terapkan seluruh manifest ke cluster
kubectl apply -k deploy/k8s

# 3. Verifikasi status pod, service, network policy, dan ingress
kubectl get pods,svc,networkpolicy,ingress -n pulseops
```

Spesifikasi Zero-Trust NetworkPolicy (`deploy/k8s/networkpolicy.yaml`)

Mengisolasi perimeter jaringan pod secara ketat: lalu lintas masuk (Ingress) hanya diizinkan pada port 8080, sedangkan lalu lintas keluar (Egress) dibatasi secara eksklusif hanya ke port 53 (DNS UDP/TCP) dan port 80/443 (HTTP/HTTPS) untuk keperluan probe target eksternal.

B. Provisioning Deklaratif Menggunakan Terraform (IaC)

Konfigurasi Terraform di `deploy/terraform/` mengotomatiskan provisioning container dan persistent storage:

```
cd deploy/terraform
terraform init
terraform plan
terraform apply -auto-approve
```

C. Deployment Cloud Publik Gratis (Render.com / PaaS)

1. Push repositori ke GitHub.
2. Buka dashboard **Render.com** → klik **New +** → pilih **Web Service**.
3. Hubungkan dengan repositori ini. Render akan otomatis mendeteksi `Dockerfile` multi-stage.
4. Pilih Free Plan dan klik **Create Web Service**. Layanan akan aktif dalam ~2 menit dengan domain HTTPS gratis dan sertifikat SSL terkelola.

13. Pernyataan Kolaborasi AI & Vibe Coding

Proyek ini dibangun melalui sinergi intensif antara insinyur manusia dan asisten kecerdasan buatan (**GitHub Copilot / Vibe Coding**). AI dimanfaatkan secara efektif untuk mengakselerasi scaffolding pola SRE cloud-native, perancangan manifest Kubernetes deklaratif, dan pembuatan mock test suite. Setiap artefak kode diverifikasi secara ketat melalui audit keamanan dan standar industri untuk menghasilkan perangkat lunak yang tangguh, aman, dan siap produksi.